

Rapport FPGA

TP1 : Synthèse VHDL sur FPGA

I) Prise en main de la carte et des outils

1) Prise en main de la carte et des outils

On nous explique que la carte sur lequel on va travailler dispose de différentes interfaces tel que des interrupteurs, des leds des connecteur pmos bouton poussoir afficheur 7 segments et une horloge à 100 mhz

2) Création d'un projet avec le logiciel Vivado

On crée notre premier projet vivado.

3) Création d'un module VHDL

On renseigne les fichiers de code et de contraintes comme demandé dans l'énoncée.

4) Testbench et simulation

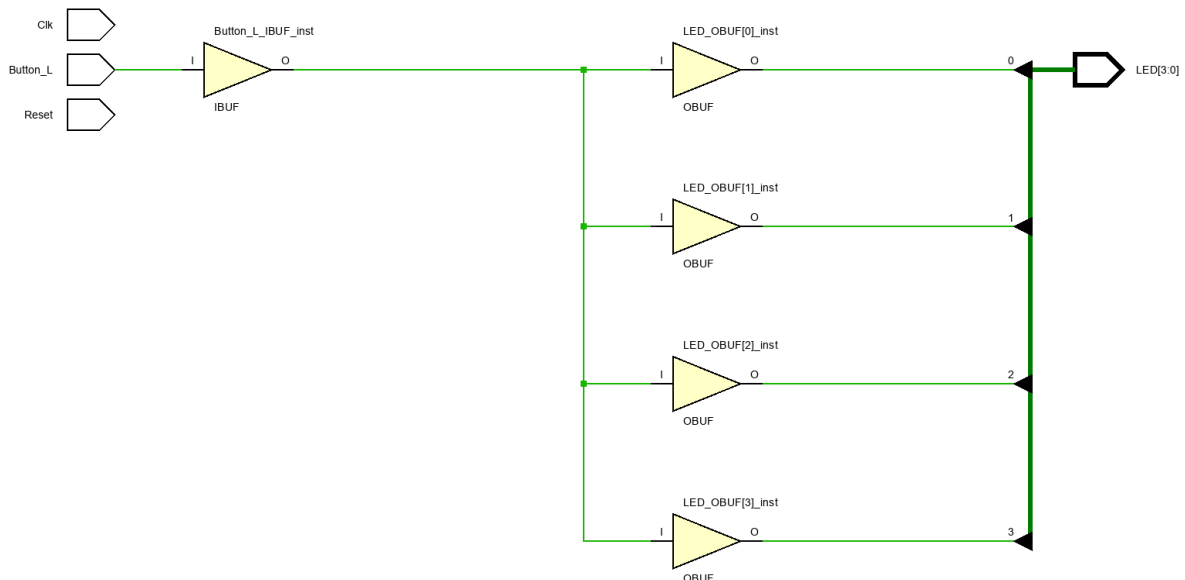
On affiche la simulation de ce dernier

Name	Value
SW0	0
SW1	1
SW2	1
> LED[2:0]	6

5) Implémentation sur la carte FPGA

Nos observations sont que quand on programme sur la carte 4 led s'allument quand on appuie sur le bouton de gauche.

Voici le schéma :



ça ne correspond pas au code de base de vhdl car il n'y a pas de compteur ni de clock seulement des connection entre les leds et boutons.

II) Cas d'études – Synthèse VHDL.

1) Compteurs Imbriqués

- Quelle est la fonctionnalité décrite dans le VHDL?

Ça d'écrit un compteur qui incrémente quand on appuie sur un bouton et s'affiche sur les led.

- A quel résultat peut-on s'attendre sur la carte ?

à ce que quand on appuie sur un bouton le compteur augmente mais ce n'est pas le cas.

on a eu le warning

[Synth 8-327] inferring latch for variable 'start_reg'

["C:/Users/21500778/Downloads/FPGA1_TP1_2024/Partie_2_Etudes_de_Cas/1_Compteurs_Imbriques/Test_CPT.vhd":43]

c'est cette ligne `if Reset = '1' then Cpt <= 0; start <='0';` qui a causée l'erreur :
"inferring latch for variable "start_reg"

on peut la régler en modifiant la valeur signal **Cpt: integer range 0 to 20000000**; en signal **Cpt: integer range 0 to 70000000**; -- Compteur.

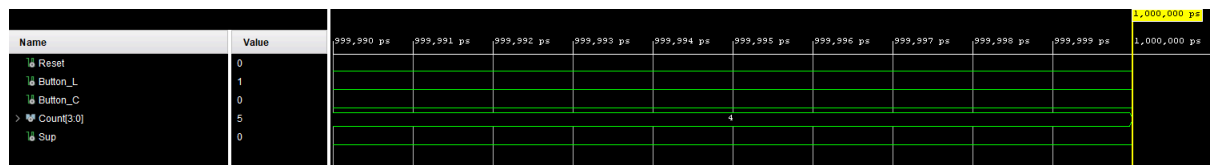
Puis, on “**start <= start**” dans la boucle

2) Compteur d’Impulsions

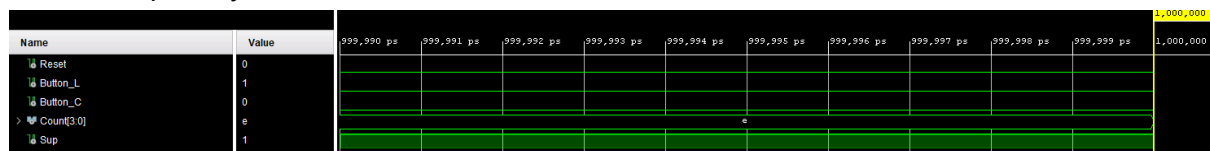
- Quelle différence y a-t-il entre la description VHDL et l’architecture synthétisée ?

Sur l’architecture on n’a qu’une sur le code vhdl on a deux horloges par boutons. Le code vhdl décrit un mécanisme où quand on appuie sur le bouton de gauche il augmente et celui du centre il diminue

Résultat des simulations:

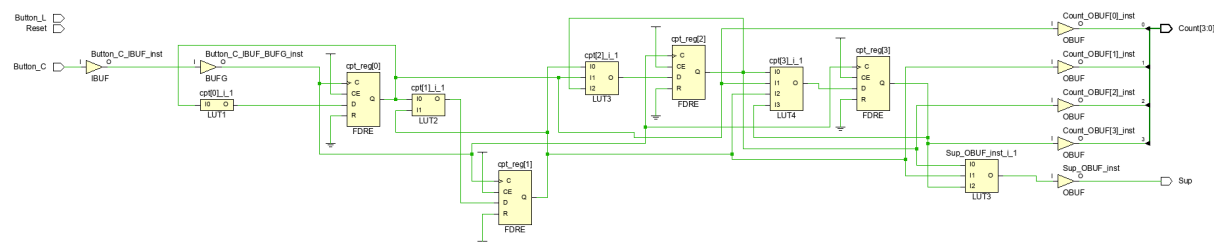


Simulation post-synthèse:



On remarque qu’il y a une case sup remplie dans le deuxième chronogramme comparé au premier.

la synthèse ne donne pas de warning et donne ce schéma :



C’est étrange car dans le code on dit que le bouton centre et gauche ont une horloge chacun donc ils devraient être tous les deux en entrée dans le schéma au moins c’est pas le cas ici.

Il ne faut pas mettre une clock par bouton mais une clock et vérifier la valeur de chaque boutons et utiliser une sorte de buffer pour compter le nombre de cycles avant qu’on puisse appuyer sur le bouton pour éviter le phénomène de rebond.

```

```

if rising_edge(Clk) then
 -- Incréméntation Si on Appuie sur le Bouton Left
 if(Button_L = '1') then

```

```
if (cooling_btn_gauche = "1000000") then
```

```
 cpt<=cpt+1;
```

```
 cooling_btn_gauche <= "0";
```

```
end if;
```

```
end if;
```

```
if (cooling_btn_gauche < 1000000) then
```

```
 cooling_btn_gauche <= cooling_btn_gauche +1;
```

```
...
```

et une erreur ici :

```
set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports { Coun[0]}]
```

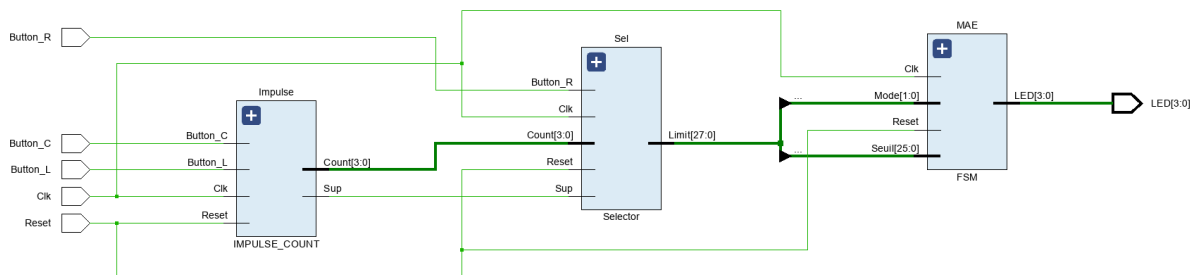
il manque le "t"

et vu qu'on utilise qu'une horloge on doit la mettre dans le test bench et ajouter la ligne

```
Clk <= not Clk after 5ns;
```

pour que ça varie

on obtient ce design :



### 3) Décodeur & Machine à États

Le fichier TOP est au haut dans la hiérarchie.

Si on appuie 3 fois sur le bouton de gauche et une sur celui de droite au lieu de clignoter les leds s'allument simplement et si on le reste avec la switch elle ne s'éteint pas complètement elles sont toujours allumées un peu.

des erreur de type inferring latch on peut être provoquée ce problème car on a 3 warning ainsi que :

"[DRC CFGBVS-1] Missing CFGBVS and CONFIG\_VOLTAGE Design Properties: Neither the CFGBVS nor CONFIG\_VOLTAGE voltage property is set in the current\_design.

Configuration bank voltage select (CFGBVS) must be set to VCCO or GND, and CONFIG\_VOLTAGE must be set to the correct configuration voltage, in order to determine the I/O voltage support for the pins in bank 0. It is suggested to specify these either using the 'Edit Device Properties' function in the GUI or directly in the XDC file using the following syntax:

```
set_property CFGBVS value1 [current_design]
```

```
#where value1 is either VCCO or GND
```

```
set_property CONFIG_VOLTAGE value2 [current_design]
```

```
#where value2 is the voltage provided to configuration bank 0
```

Refer to the device configuration user guide for more information.”

et

“[DRC PDRC-153] Gated clock check: Net Impulse/E[0] is a gated clock net sourced by a combinational pin Impulse/Decode\_reg[1]\_i\_1/O, cell Impulse/Decode\_reg[1]\_i\_1. “

il faut décommenter la ligne :

“when others => NULL;”

et ajouter la ligne :

“EF <= EP;”

après le begin du process(Cpt,EP,Mode,Seuil)

ainsi que le cas :

“when others => EF <= LED\_OFF; “

après le cas when CLIGN\_ON

pour définir tous les cas dans les “case” au cas où.

# TP2 : Codesign Matériel / Logiciel

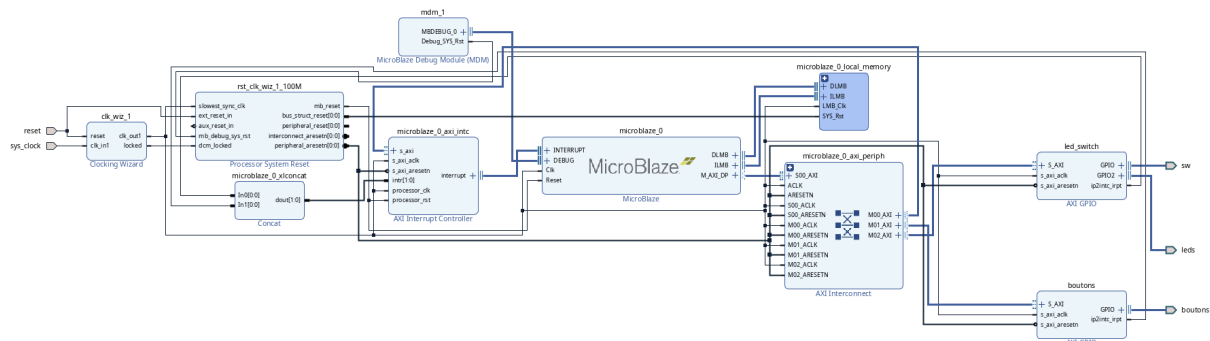
## I. Présentation

nous apprenons que Microblaze est un coeur de processeur softcore. Donc on peut modifier son fonctionnement par du code c.

- Le développement d'un système Microblaze s'effectue en 4 étapes

- Spécification de la plate-forme matérielle
- Implémentation de la plate-forme matérielle
- Création d'un projet pour la partie logicielle
- Développement de l'application logicielle

## II. Spécification de la plate-forme matérielle



Nous avons créer un nouveau projet sur Vivado, nous ouvrons le diagramme pour ajouter le module Microblaze, Clocking Wizard et Processor System Reset , Microblaze Debug Module, Microblaze Local Memory, AXI Interconnect,AXI Interrupt Controller. Nous avons ajouté 2 GPIO pour piloter les **LED, interrupteurs et boutons**. Nous avons configuré les GPIOs tels que 16 leds, 4 switches, 3 boutons sur le diagram et dans le fichier de contrainte.



LED/Switch 0, jusqu'au bit 15 pour la LED/Switch 15). De même pour les boutons : le bouton Droit correspond au masque 1, le Centre au masque 2, et le Gauche au masque 4.

Nous avons initialisé deux blocs GPIO distincts :

- **Le premier GPIO (Device ID 1)** possède deux canaux de communication (*channels*). Nous avons configuré le canal 1 en entrée (0xF) pour lire les interrupteurs, et le canal 2 en sortie (0x0) pour piloter les 16 LEDs.
- **Le second GPIO (Device ID 0)** ne possède qu'un seul canal, configuré entièrement en entrée (0xF) pour lire l'état des boutons-poussoirs.

**Architecture du Code et Variables** Le cœur de notre programme repose sur une boucle infinie `while(1)` permettant au processeur d'actualiser le système en continu. Nous avons défini plusieurs variables d'état pour gérer le système :

- `val` et `btn_state` : pour stocker la lecture brute des interrupteurs et des boutons.
- `led_actuelles` et `cpt` : pour mémoriser l'état des LEDs choisies par les boutons et la valeur du compteur.
- `blink_timer` : pour générer un clignotement asynchrone sans bloquer le programme.
- `rebond` : un temporisateur pour filtrer les appuis multiples sur les boutons.

**Le Mécanisme d'Anti-rebond** Une problématique majeure en embarqué est le rebond mécanique des boutons. Pour y pallier, nous avons créé un compteur de temporisation (rebond). Lorsqu'un bouton est pressé, cette variable prend la valeur 20 et empêche toute nouvelle lecture des boutons tant qu'elle n'est pas retombée à zéro (elle se décrémente à chaque cycle de la boucle). Couplé à un délai général `usleep`, cela laisse à l'utilisateur le temps de relâcher le bouton (environ 200 ms) sans que le compteur de LEDs ne s'incrémmente de manière incontrôlable.

**Démarche Itérative et Factorisation (La gestion des cas)** Dans la première phase de développement, le code a été conçu de manière séquentielle : chaque état d'interrupteur (Cas 1, Cas 2) possédait sa propre boucle `while` bloquante. Cependant, pour répondre au cahier des charges exigeant que les interrupteurs 0 et 1 puissent fonctionner **simultanément** (Cas 3 : compteur actif + clignotement actif), cette architecture posait problème.

J'ai donc choisi d'optimiser et de factoriser le code. Au lieu d'utiliser des boucles bloquantes, le programme :

1. Lit tous les capteurs au début du cycle.
2. Évalue de manière indépendante les conditions des interrupteurs (`if (val == 2 || val == 3)` pour le compteur, et `if ((val & 1) == 1)` pour le clignotement).
3. Additionne les résultats dans une variable temporaire `sortie_led` grâce aux opérateurs bit à bit (`|=`).

4. N'effectue qu'une **seule écriture physique** sur les LEDs à la fin du cycle, suivie d'un unique délai `usleep`.

Cette factorisation garantit un code fluide, non bloquant, et permet au processeur de traiter plusieurs tâches matérielles en "parallèle".

## Exercice 2 : Développement d'un chenillard à vitesse paramétrable

**Objectif de l'application** Dans ce second exercice, l'objectif était de concevoir un "chenillard" (un effet visuel où une seule LED allumée se déplace continuellement de gauche à droite, puis de droite à gauche) sur les 16 LEDs de la carte Basys 3. De plus, le système devait être interactif : la vitesse de défilement devait pouvoir être modifiée en temps réel grâce aux 4 premiers interrupteurs (SW0 à SW3).

**Principe d'affichage (Le décalage de bits)** Pour réaliser cet effet visuel de manière optimisée, nous avons évité d'utiliser une longue série de conditions (16 `if` successifs). À la place, nous avons exploité l'opérateur logique de décalage de bits (`<<`). En définissant une variable `position` allant de 0 à 15, l'instruction `1 << position` permet de générer dynamiquement un mot binaire ne contenant qu'un seul bit à l'état haut (par exemple `0000000000001000` pour la position 3). Ce mot est directement envoyé sur le canal 2 du GPIO des LEDs via la fonction `XGpio_DiscreteWrite`, allumant ainsi instantanément la bonne LED.

**Lecture des entrées et Masquage binaire** Le contrôle de la vitesse s'effectue via la lecture du canal 1 du GPIO relié aux interrupteurs (`XGpio_DiscreteRead`). Étant donné que le cahier des charges stipule que seuls les 4 premiers interrupteurs (SW0 à SW3) doivent piloter la vitesse, nous avons appliqué un masque binaire logique : `val_switches & 0x0F`. permet de conserver l'état des 4 premiers interrupteurs

**Gestion du mouvement et des rebonds aux extrémités** Pour animer le chenillard, le programme utilise une boucle infinie dans laquelle la variable `position` est modifiée à chaque itération. Afin de gérer le va-et-vient, nous avons introduit une variable `direction` (initialisée à 1). À la fin de chaque cycle, la position est mise à jour (`position = position + direction`). Des conditions logiques surveillent les limites physiques de la carte : si la position atteint 15 (bord gauche) ou 0 (bord droit), le signe de la direction est inversé (`direction = -1` ou `direction = 1`). Le chenillard "rebondit" ainsi naturellement sur les bords.

# TP3 : CONCEPTION D'IP POUR LE MICROBLAZE

## 1 Présentation

L'objectif de ce TP est de réaliser une IP destinée à être connectée au Microblaze, en remplacement du contrôleur de LED.

Nous apprenons que Microblaze est un cœur de processeur softcore. Donc on peut modifier son fonctionnement par du code c.

### 1) Création d'une IP Contrôleur de LED

Nous avons copier le bloc design du TP2,

On configure et on crée une IP contrôleur LED s'appelant **"my\_led\_1\_0"** crée le fichier my\_led.vhd pour le faire :

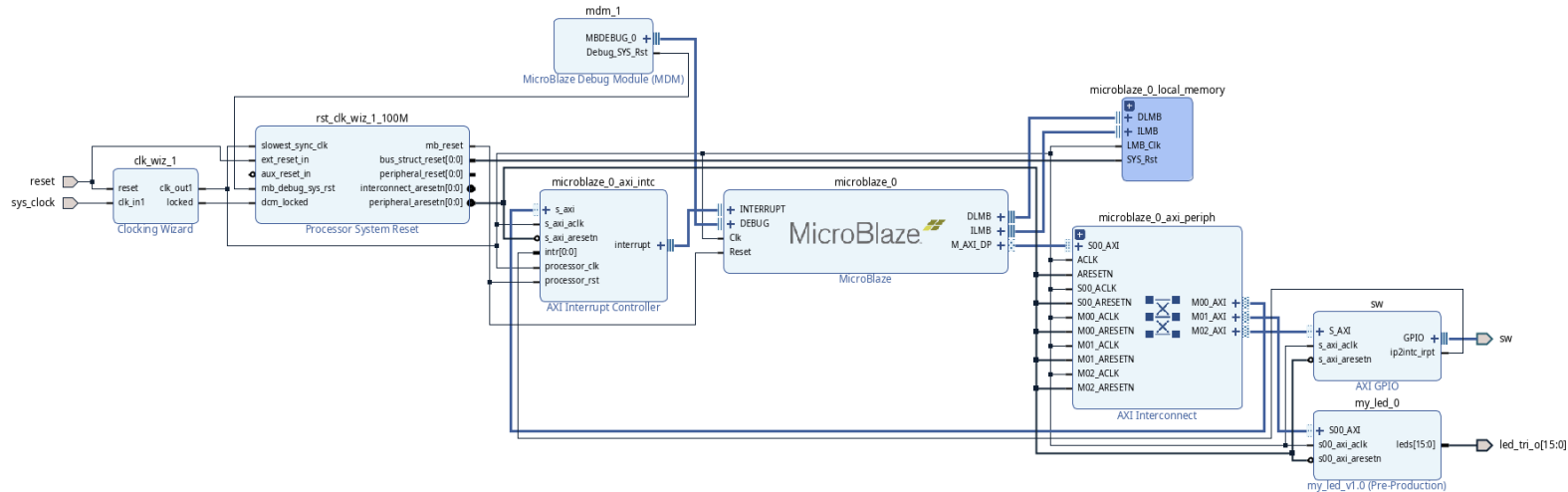
```
library ieee;
use ieee.std_logic_1164.all;
entity Led is
port(sw_state: in std_logic_vector(3 downto 0); leds : out std_logic_vector(15 to 0));
end Led;
architecture Archi of Led is
begin
 -- Si sw_state(0) = '1', on allume les LED(3:0)
 leds(3 downto 0) <= "1111" when sw_state(0) = '1' else "0000"; -- Si sw_state(1) = '1', on allume
les LED(7:4)
 leds(7 downto 4) <= "1111" when sw_state(1) = '1' else "0000"; -- Si sw_state(2) = '1', on allume
les LED(11:8)
 leds(11 downto 8) <= "1111" when sw_state(2) = '1' else "0000"; -- Si sw_state(3) = '1', on allume
les LED(15:12)
 leds(15 downto 12) <= "1111" when sw_state(3) = '1' else "0000";
end Archi;
```

ensuite on a suivi les étapes et on a changé le fichier de contraintes XDC en enlevant certaines lignes et changeant les nom pour correspondre aux bon nom de boutons et switch etc :

```
set_property PACKAGE_PIN V17 [get_ports {sw_tri_i[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_tri_i[0]}]
set_property PACKAGE_PIN V16 [get_ports {sw_tri_i[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_tri_i[1]}]
set_property PACKAGE_PIN W16 [get_ports {sw_tri_i[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_tri_i[2]}]
set_property PACKAGE_PIN W17 [get_ports {sw_tri_i[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {sw_tri_i[3]}]
```

## 2) Modification du makefile du driver C de l'IP

Dans cette étape on a du changer le fichier makefile par précaution et ensuite relier le tout (entrée sorties) avec le microblaze dans le schéma :



## 3) Intégration de l'IP au système Microblaze

Puis ici on a fait le code vhdl du wrapper

```
entity TP3_wrapper is
 port (led_tri_o : out STD_LOGIC_VECTOR (15 downto 0);reset : in STD_LOGIC;sw_tri_i : in
STD_LOGIC_VECTOR (3 downto 0);sys_clock : in STD_LOGIC);
end TP3_wrapper;
architecture STRUCTURE of TP3_wrapper is
 component TP3 is
 port (led_tri_o : out STD_LOGIC_VECTOR (15 downto 0);sw_tri_i : in STD_LOGIC_VECTOR (3
downto 0);sys_clock : in STD_LOGIC;reset : in STD_LOGIC);
 end component TP3;
begin
 TP3_i: component TP3
 port map (led_tri_o(15 downto 0) => led_tri_o(15 downto 0),reset => reset,sw_tri_i(3 downto 0)
=> sw_tri_i(3 downto 0),sys_clock => sys_clock
);
end STRUCTURE;
```

## 4) Développement logiciel

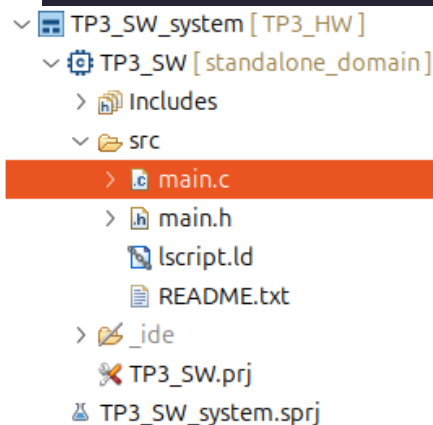
Nous avons pu constater que le fichier my\_led.h s'est bien généré.

Ce qui nous a permis de faire le fichier main.c avec le code suivant

```
Include Files *****/
#include "my_led.h"
#include "xil_io.h"
#include "xparameters.h"
#include "xgpio.h"

/***** Function Definitions *****/

int main(){
 XGpio sw_gpio;
 u32 sw_state;
 u32 reg0_data, reg1_data;
 XGpio_Initialize(&sw_gpio, XPAR_SW_DEVICE_ID);
 XGpio_SetDataDirection(&sw_gpio, 1, 0xF);
 while(1){
 sw_state = XGpio_DiscreteRead(&sw_gpio, 1);
 reg0_data = 0;
 reg1_data = 0;
 // On isole le bit 0 et le bit 1 des interrupteurs pour
slv_reg0
 reg0_data |= sw_state & 0x03;
 // On isole le bit 2 et le bit 3 des interrupteurs et on decale
pour slv_reg1
 reg1_data |= (sw_state >> 2) & 0x03;
 MY_LED_mWriteReg(XPAR_MY_LED_0_S00_AXI_BASEADDR,
MY_LED_S00_AXI_SLV_REG0_OFFSET, reg0_data);
 MY_LED_mWriteReg(XPAR_MY_LED_0_S00_AXI_BASEADDR,
MY_LED_S00_AXI_SLV_REG1_OFFSET, reg1_data);
 }
 return 0;
}
```



On a pu créer et placer notre main.c

dans les sources du projet sur vitis.

Le build s'est fait le débogage à suivi et enfin l'exécution a été réalisée sur le matériel.

### Explication du main.c :

Inclusion de la bibliothèque "my\_led.h".

Initialisation et déclaration du gpio switch, on configure son canal vers entrée.

### Boucle Principale de Traitement (while(1)) :

- Le programme lit l'état complet des 4 interrupteurs. Si les 4 sont activés, la valeur sera 0xF (1111 en binaire).
- **Traitement et Masquage des bits** : Le code divise les 4 interrupteurs en deux paires distinctes à l'aide d'opérations bit à bit :
  - `reg0_data |= sw_state & 0x03;` : On applique un masque logique "ET" avec 0x03 (0011 en binaire). Cela permet de conserver uniquement l'état des **interrupteurs 0 et 1**, en mettant les autres bits à zéro.
  - `reg1_data |= (sw_state >> 2) & 0x03;` : On décale d'abord la valeur lue de 2 bits vers la droite (>> 2), ce qui déplace les **interrupteurs 2 et 3** vers les positions 0 et 1. On applique ensuite le même masque 0x03 pour isoler ces deux bits.
- **Écriture dans les registres du composant AXI (IP) :**
  - `MY_LED_mWriteReg()` : Ces deux lignes utilisent une macro pour envoyer les données traitées via le bus AXI.
  - La valeur de `reg0_data` (interrupteurs 0 et 1) est écrite dans le **Registre Esclave 0** (SLV\_REG0) de l'IP MY\_LED.
  - La valeur de `reg1_data` (interrupteurs 2 et 3) est écrite dans le **Registre Esclave 1** (SLV\_REG1).